

Ch1 初识C语言与程序框架

对应《C Primer Plus（第6版）》第 1-2 章合并 · 本章目标：30 分钟读懂 + 60 分钟练完 · 不覆盖：指针、位运算

1.1 学 C 之前：这语言为什么值得学

C 语言是很多大学电子信息/计算机专业的第一门编程课。它离"硬件"很近——你能直接控制内存地址、字节、硬件寄存器，这是 Python/Java 这些"更省事"的语言替你藏起来的东西。兰大的电子信息课程里，单片机的 C 代码、嵌入式、信号处理库，背后都是 C。

一句话定位：C 让你看清"程序是怎么跑起来的"。哪怕以后主要用别的语言，C 建立的"内存、指针、编译"心智模型也会跟着你走。

但这门课有 **两座大山**：指针（第 10 章）和内存管理（第 12 章）。别怕——本讲义会在前面章节不断埋伏笔，到那两章你就不觉得陡了。

1.2 环境搭建：让"编译器"跑起来

你写的 C 代码（.c 文件）计算机不能直接运行，需要先用**编译器**把它翻译成机器能懂的机器码。本课程统一用 **MinGW-W64 的 gcc**（本机已装，16.1.0）。

三步走：

1. **写代码：**用编辑器（VS Code / Dev-C++ / 记事本都行）写一个 hello.c 文件，纯文本保存；
2. **编译：**命令行敲 `gcc -Wall -Wextra -o hello.exe hello.c`，生成可执行文件 hello.exe；
3. **运行：**敲 `.\hello.exe`，看到输出。

```
/* 编译 + 运行（在 hello.c 所在目录的终端里敲）*/  
gcc -Wall -Wextra -o hello.exe hello.c /* 生成 hello.exe */  
.\hello.exe                          /* 运行，看到 "Hello, C!" */
```

为什么要 -Wall -Wextra？ 它让编译器把「可疑但还没到报错」的写法也提示出来（叫"警告"，warning）。初学者的 90% 低级错误——变量没初始化、类型不匹配、变量没用到——靠这两个选项提前暴露。养成习惯：写代码就开这两个选项，看到警告先修掉再运行，别"能跑就行"。

1.3 第一个程序：Hello, C!

每一个 C 程序都长这样（先背下来，后面所有章节都在这个骨架里加东西）：

```
/* ch1_01_hello.c - 第一个程序 */
#include <stdio.h>          /* 引入"标准输入输出"库：printf/scanf 都在这里 */

int main(void)              /* 程序入口：程序从这里开始执行 */
{
    printf("Hello, C!\n");  /* 输出一行文字；\n 是换行 */
    return 0;              /* 告诉操作系统"正常结束" */
}
```

编译: gcc -Wall -Wextra -o hello.exe hello.c 运行: .\hello.exe → 输出: Hello, C!

四个部件逐个看：

- `#include <stdio.h>`：告诉编译器"我要用标准输入输出函数"。stdio.h 是"标准头文件"，里面声明了 printf/scanf 等。不 include 就用 printf，编译器会报"隐式声明"错误。
- `int main(void)`：每个程序必须有且只有一个 main，它是入口。int 表示 main 会返回一个整数，void 表示"不接受参数"。
- `printf(...)`：打印函数。\\n 是"换行"转义字符。
- `return 0;`：返回 0 表示程序正常结束。返回非 0 通常表示出错。

1.4 编译错误三行对照：先学会读报错

零基础最先崩溃的不是写错逻辑，而是看不懂编译器的报错。养成习惯：看到报错，先看行号和报错关键字，再去那一行附近找问题。

报错原文 (gcc 常见)	人话	怎么改
error: expected '=', ',', ';', 'asm' or '__attribute__' before 'printf'	少了分号	上一行末尾补；（同类错误在不同上下文提示文字不同——此为本题实测原句）
error: 'x' undeclared (first use...)	变量 x 没声明	用之前先 <code>int x;</code> 声明
warning: unused variable 'x'	x 声明了没用	删掉，或真用起来
error: implicit declaration of function 'printf'	printf 没声明	检查 <code>#include <stdio.h></code> 是否写上（GCC 14 起为 error，旧版仅 warning——本机 gcc 16.1.0 实测为 error）
undefined reference to 'main'	找不到 main	确认有 <code>int main(void)</code>

新手坑 1：报错行号只看个大概。gcc 报的行号往往指向问题"所在的那一行附近"，真正的错误可能在前一行（比如漏了分号时，报错常落在下一行的）。从报错行往上多看两行，别死盯那一行。

1.5 数据和变量：程序怎么"记住"东西

程序要处理数据，就得先把数据放到"盒子"里——这个盒子叫**变量**。用之前必须先**声明**：告诉编译器"我要一个装整数的盒子，叫 num"。

```
/* ch1_02_var.c - 声明、赋值、输出 */
#include <stdio.h>

int main(void)
{
    int age;          /* 声明：一个装 int（整数）的盒子，名叫 age */
    age = 19;         /* 赋值：往盒子里放 19 */

    int score = 95;   /* 声明 + 赋值，一步到位（初始化）*/

    printf("我 %d 岁，这次考了 %d 分\n", age, score);
    return 0;
}
```

运行输出：我 19 岁，这次考了 95 分

常用的基础类型先认识三个（更多在第 3 章 Ch3）：

类型	含义	示例
int	整数	int n = 42;
double	双精度小数	double pi = 3.14159;
char	单个字符	char c = 'A';

新手坑 2：变量没初始化就用。声明了变量但没赋值（没放东西）就用，它的值是**内存里的残留值**——不确定、可能很大或很怪。这就是为什么 `-Wall -Wextra` 会警告 `uninitialized`。入门规矩：声明时就顺手给个初值（`int score = 95;`）。

1.6 新手坑集中营（本章最容易错的 3 件事）

1. **漏分号**：C 里每条"语句"（`printf`、赋值、`return`）结尾都要 `;`。漏了编译器从下一行才反应过来，报错看起来"错在别处"（见 1.4 对照表第一行）。
2. **变量没初始化就用**（见 1.5）——开 `-Wall` 让编译器替你抓。

3. 忘了 **return 0**：C99 起 main 结尾缺 `return 0`；编译器会补，但养成显式写的习惯，也避免有些环境告警。

1.7 一分钟自测（答案见练习册）

判断对错：

- ① 程序必须先 `#include <stdio.h>` 才能用 `printf`。
- ② `int n`；声明后，`n` 的初始值一定是 0。
- ③ `main` 是程序入口，每个程序只能有一个。

1.8 练习指引

做完本章内容后，去练习册完成 Ch1 配套练习；同时打开《C Primer Plus（第6版）》第 1、2 章，完成**复习题 1、2、3**（题号对应主教材）。改错题是练习册里最有价值的部分——先自己找 bug，再看答案的思路讲解。

本讲义为自写原创讲解，仅作课堂伴读；示例代码均可用 `gcc -Wall -Wextra` 编译。原书题目请以正版教材为准。